Trend Analysis of the Automated Aeroponics Laboratory System Slimsgate Plugin

Anthem Rukiya J. Wingate

Stevens Institute of Technology
School of Systems and Enterprises

"I pledge on my honor that I have not given or received any unauthorized assistance on this assignment/examination. I further pledge that I have not copied any material from a book, article, the Internet or any other source except where I have expressly cited the source."

Abstract

It is critical to determine protype viability throughout the software development lifecycle of the AALS. The team will validate and verify the performance of each iterative prototype of the AALS. The team will analyze the time to implement each solution and the time to compile the completed solution for a quantifiable determination of prototype viability. Time to complete and time to compile will be framed as metrics using the GQM process. Data collection and statistical validation of the results will be conducted using tools outlined in the overview of this paper. The measurements observed for each implementation are discussed and reflections on the implications of these observations are composed to support broad scale conclusions determining the viability of the prototype. Discussion of the outcomes lends attention to the limitations encountered by the team while completing this study and finally draws conclusions which briefly reiterate findings and offer insight to potential future utilization and impact of this development pathway.

Keywords: Java, Python, No-Code, GQM

Table of Contents

1. Overview
    a. Introduction
    b. Relevance of Study
    c. Methodology
    d. Data Collection
    e. Dataset Variables
    f. Dataset Resource
    g. Tools
2. Metric Validation

   a. Goals
   b. Questions
   c. Metrics
   d. Mechanism
3. Results and Discussion
   a. Data
   b. Limitations
   c. Reflection
   d. Conclusion
4. References
5. Appendices
   a. Source Code
   b. List of Tables
   c. List of Figures

Amid a pandemic-induced declining global economy, it is notable that software companies are experiencing profit growth and facing increased demand. The implication of this unexpected behavior in the market is that there is an answering provision of high performing product releases to respond to ever-increasing demand. In this project conducted by SSW 690A Fall 2020 Aero Team, the performance metrics of prototypes of the AALS Slimsgate plugin are studied in order to make a quantitative determination of longevity and efficiency.

Relevance of Study

This study has constructed a means by which to model quantitative comparative analyses between prototypes of the AALS Slimsgate plugin. This project seeks to validate comparisons in performance of these prototypes by implementing the application of a quantifiable testing methodology.

Methodology[1]

There are two metrics that have been identified to complete this study. The time to implement the prototype, i.e. "solution" and the time to compile the solution will be recorded in observation and examined in order to draw conclusions illuminating the efficiency of iterative versions of the AALS Slimsgate plugin built and committed to the Github platform. The Goals, Questions, Methods Process (GQM) has been implemented to identify these metrics.

Data Collection. Project data has been collected via metrics compiled for each prototype at time of build. These metrics include a code review of the solution lending particular focus to cyclomatic complexity, shared arguments and operators, and code coverage. Shared arguments and operators in overlapping solution architectures will serve as dataset variables intended to establish correlation between each prototype in order that a valid comparison may be performed using chosen metrics.

Dataset Variables.

The handlers that are shared in common between each prototype are listed in the following tables. These beans are comparable due to commonality of use in solution and relative complexity of solution. Cyclomatic complexity is determined using the following formula:

$$CYC = E - N + 2P$$

P = Functions
E = Libraries
N = Variables

The prototype built in commit `f196f7052a17e0abe13dc3fb3006a68966a8d35b` was found to have a cyclomatic complexity of 40 The prototype built in commit `612828d6e36467fbb6be290d2a444736eaf68a8e` was found to have a cyclomatic complexity of 43 The prototype built in commit `b2e1c945019689861b3fc2d5b60423fcf62f772f` was found to have a cyclomatic complexity of 55. Though the cyclomatic complexity is fairly close between the first and second commit, it is substantially different between the second and third commit. This would indicate that a difference in either the number or level of detail in the requirements implemented during the first and the third commits. In addition, one may conclude that no new requirements

were implemented during the second commit. Supporting evidence of these conclusions were drawn in examination of the Kanban workflows outlined during corresponding Heartbeat Meetings during the weeks leading up to each commit.

Prototype by Commit	**AALSBuildAndOutputEmailsBean**	**AALSCreateContentBean**	**AALSHandleActiveUserBean**	**AALSMapJSONBean**	**AALSResponseBean**
`f196f7052a17e0abe13dc3fb3006a68966a8d35b`	Functions: 2	Functions: 1	Functions: 3	Functions: 2	Functions: null
	Libraries: 21	Libraries: 8	Libraries: 7	Libraries: 13	Libraries: null
	Variables: 9	Variables: 2	Variables: 2	Variables: 5	Variables: null
	LOC: 64	LOC: 39	LOC: 36	LOC: 49	LOC: 0

Table 1.1

Prototype by Commit	**AALSBuildAndOutputEmailsBean**	**AALSCreateContentBean**	**AALSHandleActiveUserBean**	**AALSMapJSONBean**	**AALSResponseBean**
`612828d6e36467fbb6be290d2a444736eaf68a8e`	Functions: 2	Functions: 1	Functions: 3	Functions: 2	Functions: null
	Libraries: 21	Libraries: 8	Libraries: 7	Libraries: 13	Libraries: null
	Variables: 9	Variables: 2	Variables: 2	Variables: 5	Variables: null
	LOC: 64	LOC: 39	LOC: 36	LOC: 49	LOC: 0

Table 1.2

Prototype by Commit	**AALSBuildAndOutputEmailsBean**	**AALSCreateContentBean**	**AALSHandleActiveUserBean**	**AALSMapJSONBean**	**AALSResponseBean**
`b2e1c945019689861b3fc2d5b60423fcf62f772f`	Functions: 2	Functions: 1	Functions: 3	Functions: 2	Functions: 3
	Libraries: 21	Libraries: 9	Libraries: 7	Libraries: 11	Libraries: 10
	Variables: 9	Variables: 3	Variables: 2	Variables: 5	Variables: 8
	LOC: 69	LOC: 40	LOC: 36	LOC: 49	LOC: 61

Table 1.3

Dataset Resource. The datasets collected in this study are sourced from the implementation of a solution for a PNG. The code for the PNG was architected using Visual Studio Code and managed using Github. During the development process, builds of the PNG were continuously integrated using Travis CI and analyzed for Coverage using CodeCov.IO.

Tools.

GitHub

A product of Microsoft Organization, web-based private and public repositories for all levels of collaborative software development.

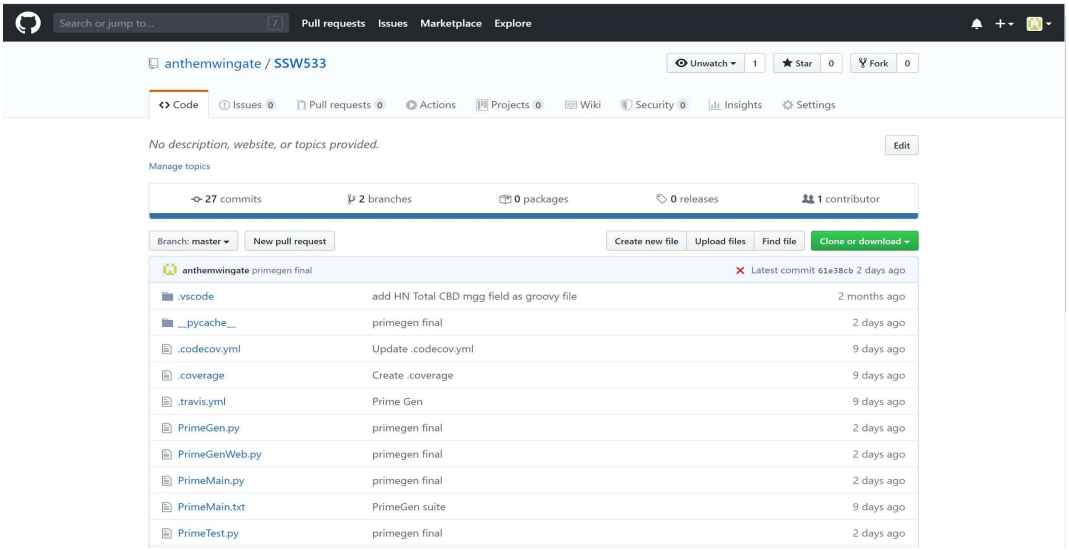

Figure 1

IntelliJ

Is an integrated development environment developed by JetBrains.

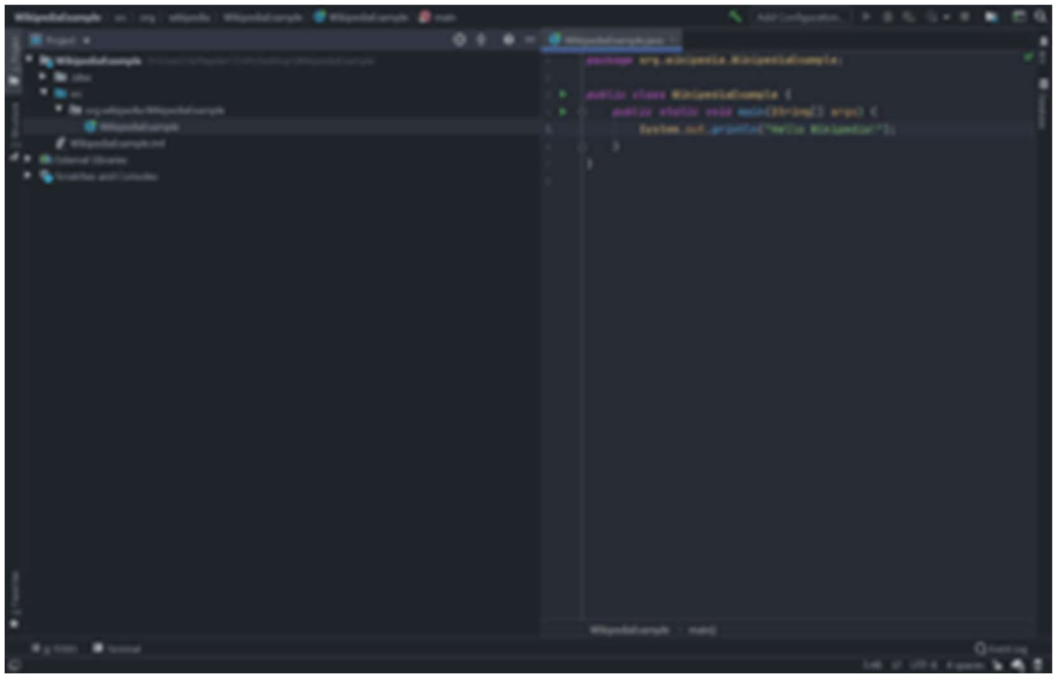

Figure 2

Jira

A product of developed by Atlassian for Agile project management and issue tracking.

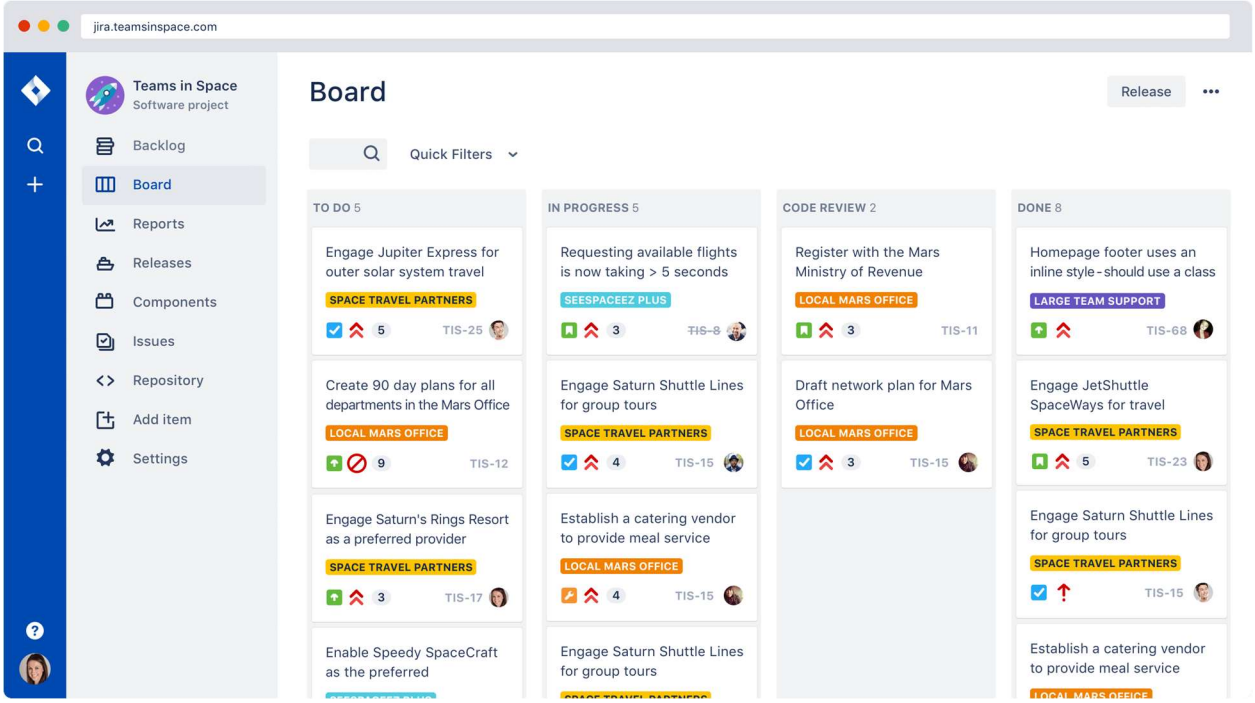

Figure 3

Postman

An opensource collaboration platform for API development .

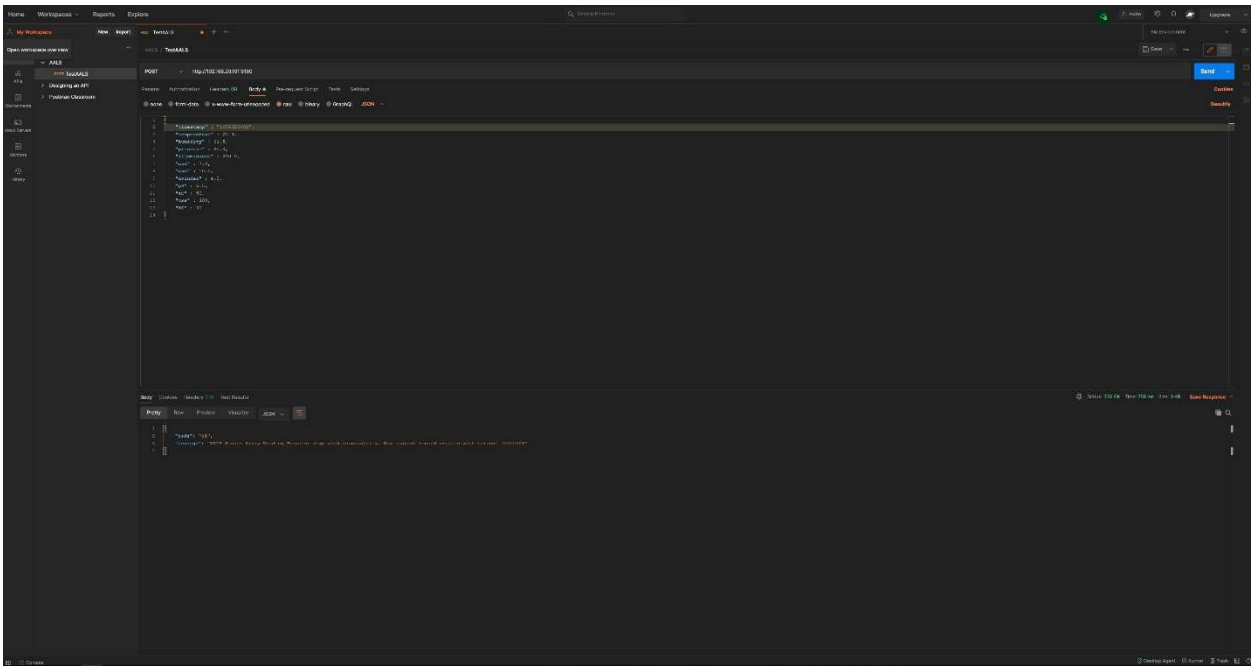

Figure 4

SLIMS

A simplified laboratory informatics management system.

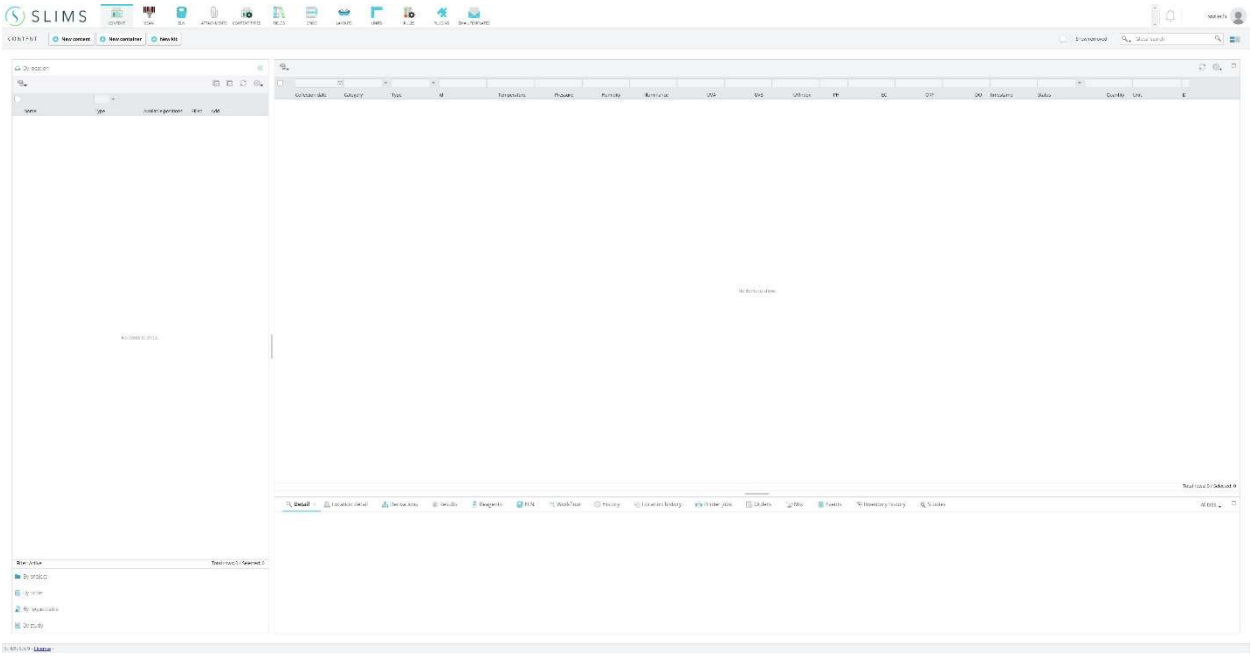

Figure 5

Metric Validation[2] The purpose of this study is to observe and verify through measurement the viability of each prototype based on performance. "Every company that builds products to sell is concerned with time to market" (Jones 2007). "Process or internal innovations, involve developing new or improved methods of development that can shorten development times, reduce costs, or improve quality" (Jones 2007). The metrics chosen in this GQM process were identified in accordance with a key business driver - development time.

Time to Implement Solution. The proposed metric is a measurement of the duration of time expended between commits for each iterative prototyping cycle. Each week day within the number of days represents an accrual of four hours development time including time allocated for research.

Goals. The goal of this metric is to provide precise measurement for the time required for a developer to architect for a deepening pool of functional requirements. Data acquired in these observations may be analyzed in order to compare relative performance of the prototype as requirements become more detailed and the project progresses toward product release.

Questions. How much time would be required for the average programmer to architect a solution for based on level of detail of corresponding functional requirements? How does level of detail of the requirements being architected impact the time expended architecting a solution?

Metrics. The time to architect was recorded and broken up into two categories. These categories are listed based on effort utilized for research and effort utilized for coding. These categories are intended to shape a more accurate picture of the distribution of effort across the total time to architect the solution by providing key insight into how much time was spent researching

how to get the solution and building it. It is the expectation of this study that the distribution of effort will vary dependent on the relative level of detail of the functional requirements, i.e. low, moderate, or high in correspondence with the first, second, and third build commit.

Mechanisms.

Commit	Time researching (hours:minutes)	Time programming (hours:minutes)
Commit ID 1		
Commit ID 2		
Commit ID 3		

Table 2

Time to Compile Solution. The proposed metric is a measurement of the duration of time required for the compiler to run the program. The compiler converts the solution into machine code.

Goals. The goal of this metric is to provide a precise measurement of the time required to compile a successful build for the prototype. Data acquired in these observations may be analyzed in order to compare relative performance of each prototype.

Questions. How do prototypes with increasingly more detailed requirements compare based on speed of program execution?

Metrics. The average time of 5 test executions for which the time to compile was recorded.

[T1, T2, T3, T4, T5: n = 5]

Each test was recorded as variable T. Incrementing the subscript for each consecutive test from one to five, yields the variables representing each recorded observation for the time to compile. The average time to compile was then calculated by taking the sum of the five test executions, *T1:T5* and dividing by the total number of tests, n.

$(T1+T2+T3+T4+T5)/5=T_{avg}$

Mechanisms.

Test	1	2	3	4	5	AVG (ms)
Compile Time	*T1*	*T2*	*T3*	*T4*	*T5*	T_{avg}

Table 3

Results and Discussions[3]

Time to Implement.

Commit	Time researching (hours:minutes)	Time programming (hours:minutes)
`f196f7052a17e0abe13dc3fb3006a68966a8d35b`	16 hr : 00 min	48 hr : 00 min
`612828d6e36467fbb6be290d2a444736eaf68a8e`	0 hr : 00 min	4 hr : 00 min
`b2e1c945019689861b3fc2d5b60423fcf62f772f`	7 hr : 00 min	21 hr : 00 min

Table 4

Time to Compile. `f196f7052a17e0abe13dc3fb3006a68966a8d35b`

Test	1	2	3	4	5	AVG (ms)
Compile Time	Null	Null	Null	Null	Null	Null

Table 5.1

Time to Compile. `612828d6e36467fbb6be290d2a444736eaf68a8e`

Test	1	2	3	4	5	AVG (ms)
Compile Time	Null	Null	Null	Null	Null	Null

Table 5.2

Time to Compile. `b2e1c945019689861b3fc2d5b60423fcf62f772f`

Test	1	2	3	4	5	AVG (ms)
Compile Time	13	2	2	4	2	4.6

Table 5.3

Discussion. Time to implement for the second commit, ID `612828d6e36467fbb6be290d2a444736eaf68a8e`, was more expedient in comparison to the first and final commit. However, the time to compile for Commit 3 was the only relevant build as the build for the first and second prototype both failed. The expedited time to implement demonstrated by the first commit implies an inversely proportional relationship between effort and performance which is

contradicted in view of the Time to Implement for the third commit paired with the successful build. Based on these numbers, time to commit is not by itself a sufficient metric to guage performance. Independently, these metrics are neither substantive nor interchangeable. The study may only suffice as a means of tracking development activities though not as a means to design more efficient development pathways for project managers seeking better overall performance.

Limitations. *What is lost with quantitative analysis of prototype performance?*

Nuance is lost with this approach to analysis of performance. There are a multitude of factors which might further mitigate the performance of each iterative prototype that are not addressed here. As an example, requirements may have been more complex in the first build than in the second. In addition, the third build may have been a simple restructuring of existing code with negligible time spent on actual implementation of new requirements.

Reflections. *What is gained with quantitative analysis of prototype performance?*

These metrics provide insight into the development pathway for those contributing to the code base. A developer seeking to better understand their capabilities might use these metrics to determine what their characteristic levels of production are based on time utilized and time to compile. Identifying successful builds to compare relative cyclomatic complexity in comparison to compile time adds a level of dimension to understanding successful patterns when architecting solutions for specific requirements.

Conclusion. Quantitative analysis of these performance related metrics using a GQM approach to the implementation of functional requirements cannot be directly correlated to trends of build success. These metrics require additional variables such as number or requirements implemented and level of detail of each requirement implemented to gain sufficient insight into any perceived trend in behavior. At present, there are no definitive conclusions that might yield predictions as to the future performance of prototypes constructed in subsequent phases.

References[4]

Jones, C. (2007). *Software Engineering Best Practices (Measurements, Metrics And Industry Leadership article)*, New York: McGraw Hill

Department of the Airforce Software Technology Support Center (2003). *Guidelines for Successful Acquisition and Management of Software-Intensive Systems: Weapon Systems Command and Control Systems Management Information Systems (Condensed GSAM Handbook Ch. 8),* Hill AFB: USAF.

Appendices[5]

Source Code.
https://github.com/Automated-Aeroponics-Innovations

List of Figures.

Figure 1	GitHub Screenshot, pg. 5
Figure 2	IntelliJ, pg. 5
Figure 3	Jira, pg. 6
Figure 4	Postman, pg. 6
Figure 5	SLIMS, pg. 7

List of Tables.

Table 1.1, 1.2, 1.3	Dataset Variable, pg. 3-4
Table 2	Example of Time to Implement data capture mechanism, pg. 8
Table 3	Example of Time to Compile data capture mechanism, pg. 9
Table 4	Time to Implement data capture mechanism, pg. 9
Table 5.1, 5.2, 5.3	Time to Compile data capture mechanism for each Github Commit, pg. 9